

전기차 배터리 화재·과열·이상 감지 — C++ / Python

전기차 배터리 화재·과열·이상 감지 시스템에서, 실시간 연산(C++) 과 영속화·외부 통합 (Python) 을 분리한 이중 서버 백엔드를 설계·구현했습니다. 두 서버가 동일한 운영 가설 — “최신 상태 우선, 손실 허용, 영구 다운 금지” — 을 각기 다른 언어로 일관되게 구현합니다.

본 문서는 포트폴리오용 PDF입니다. 회사·배포자·SDK·인프라 식별 정보와 민감 구현 세부 (레지스터 맵, DB 스키마 등)는 제거하였습니다.

1. 프로젝트 개요

항목	내용
도메인	열화상 카메라 다중 운영 기반 실시간 화재·이상 감지
구성	C++ 연산 서버 + Python 관리 서버 (2-프로세스 아키텍처)
담당 역할	Python 관리 서버 설계·구현 전담 / C++ 연산 서버 구조적 리팩토링 방향 설계
규모	현장 기준 수십 대 카메라 동시 운영

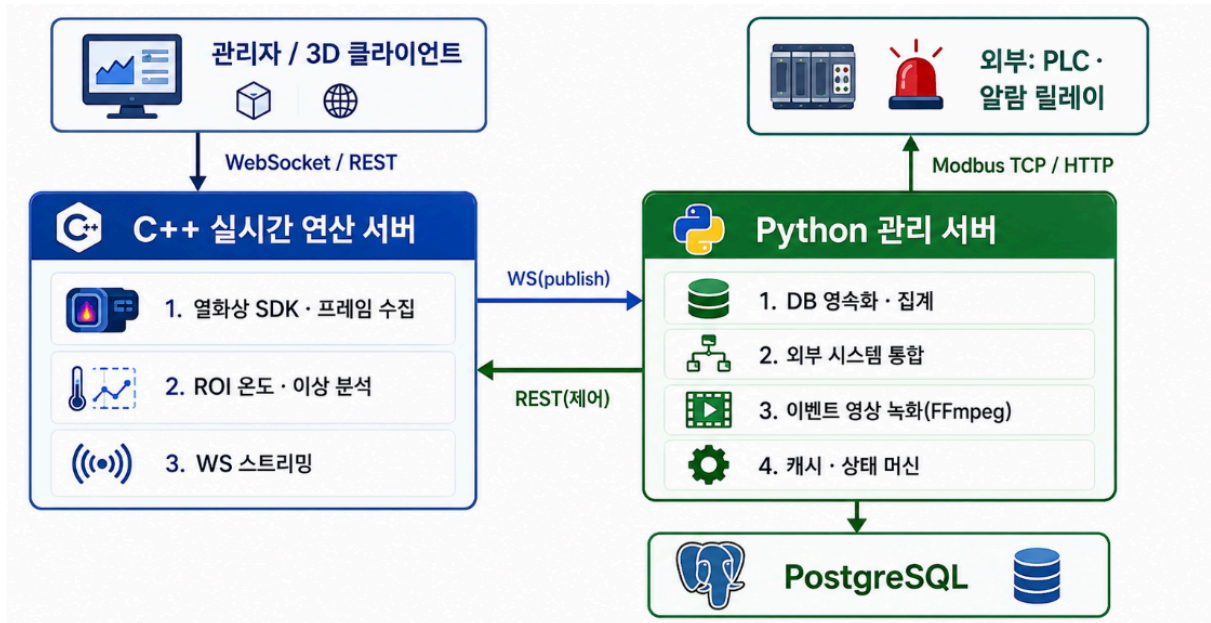
2. 기술 스택

C++ 연산 서버 — C++20 , Boost.Asio (coroutine / strand)

Python 관리 서버 — FastAPI , asyncio , SQLAlchemy , Pydantic , psycpg2 , websockets , FFmpeg (subprocess) , ctypes (Win32 API)

Database — PostgreSQL

3. 시스템 아키텍처 (개념)



- 데이터 평면: C++ → Python, WebSocket publish (이벤트 push)
- 제어 평면: Python → C++, REST (카메라/센서 connect-reconnect, ROI reload)

Part 1 — C++ 실시간 연산 서버: 구조적 리팩토링 (설계·방향)

구버전의 문제는 개별 버그가 아니라 계속해서 누적된 구조적 결함이었습니다. 패치 단위 대응은 회귀 위험이 커서, 결함을 아키텍처 수준에서 제거하는 재설계 방향을 도출했습니다

4.1 공유 상태 ↔ 락의 1:1 매칭 + 카메라별 독립 락

- 문제: 다수의 공유 맵(검출 상태·타임스탬프)이 뮤텍스 보호 없이 여러 스레드에서 접근 → data race, 심하면 rehash로 인한 dangling 참조·access violation.
- 방향: 공유 상태를 카테고리별 뮤텍스로 나눈 단일 컨테이너로 응집하고, 카메라별로 독립된 락을 부여. 카메라 A의 갱신이 B의 스트림 송신을 막지 않도록 함.
- 효과: 카메라가 늘어도 락 객체도 함께 늘어 선형 확장. race를 구조적으로 제거.

4.2 소유권 모델 재설계 — dangling·순환 참조 동시 제거

- 문제: 양방향 `shared_ptr` 순환 참조로 누수 발생. 또한 락 객체 자체가 컨테이너에서 erase되어 비동기 콜백이 use-after-free.
- 방향: 단방향 소유 트리를 강제하고, 콜백은 필요한 `shared_ptr` 만 캡처. 카메라별 상태는 `shared_ptr` 로 공유 소유해, 콜백이 살아있는 한 락도 살아있음을 보장

(`enable_shared_from_this`).

- **효과:** “누가 누구를 살려두는가”가 명확해져 lifetime 추적이 가능해짐.

4.3 단일 락 통합으로 lock-ordering·deadlock 제거

- **문제:** 함수마다 두 종류 락의 획득 순서가 달라(A → B vs B → A) deadlock 가능. 재현이 어렵고 발생 시 서버 전체 정지.
- **방향:** 도메인 데이터를 응집해 **카메라별 단일 락**으로 통합 → 락이 하나면 순서 자체가 존재하지 않음.

4.4 락프리 백프레서

- **문제:** 송신·분석 큐에 상한이 없어, 느린 클라이언트나 처리 지연 시 메모리 무한 증가. 이전 프레임 처리 중에도 작업이 중첩 제출됨.
- **방향:** WS 송신 큐에 상한(오래된 프레임 자동 폐기 + 드롭 카운터), ROI 분석에는 카메라별 in-flight 플래그를 `compare_exchange_strong` 으로 **락프리 스킵**. lock contention 없이 백프레서 구현.

4.5 IO/CPU 분리 — 3개 전용 스레드 풀

- **문제:** 단일 io_context·단일 풀에서 ROI 분석(CPU 집약)이 네트워크 응답성을 잠식.
- **방향:** **IO · ROI · 인코딩** 3개 전용 풀로 분리(`ioThreads/roiThreads/encodeThreads` 를 코어 수 기반 산정). CPU 작업이 네트워크 스레드를 굼기지 못하게 함.

4.6 콜백 지옥 → C++20 코루틴 + strand

- **문제:** 람다 중첩(lambda in lambda)으로 lifetime 추적·캡처 실수 위험. 여러 소스가 같은 WS에 동시 write → protocol violation.
- **방향:** `co_spawn` / `co_await` 으로 비동기 로직을 **선형 코드**로 전환. 세션마다 `asio::strand` 를 두어 write를 자동 직렬화.

4.7 장시간 동기 호출 중 글로벌 락 점유 해소

- **문제:** SDK 동기 connect가 TCP timeout(수십 초)까지 반환하지 않을 때 글로벌 락을 잡고 있으면, 그동안 **다른 모든 카메라가 마비**.
- **방향:** SDK 호출 직전에 `unique_lock` 을 해제하고, 진행 중인 이전 재연결은 **atomic abort** 토큰으로 중단 신호를 전달.

4.8 God class 해체 — 책임별 모듈 분리

- **문제:** 한 클래스가 라이프사이클·ROI·검출·WS·큐·저장 등 6+ 책임을 가져 변경 영향이 폭발하고, 락 종류 증가·테스트 불가.
- **방향:** 라이프사이클 / ROI 워커 / WS 허브·세션 / 주기 작업(코루틴) / DB / 공유 상태로 **책임별 분리**. 계층은 `common → hal → net.pipeline` 의 한 방향 의존만 허용해 순환을 원천 차단.

Part 2 — Python 관리 서버: 영속화·통합 (설계·구현 전담)

C++만으로 영속화·집계·외부 통합·녹화까지 끌고 가는 것은 개발 속도·운영 안전성 면에서 비용이 컸습니다. 이 책임들을 별도 FastAPI 서버로 분리해 전담 구현했습니다.

5.1 async hub → Queue → sync DB worker 분리

- **문제:** asyncio 안에서 동기 SQLAlchemy를 직접 호출하면 그동안 **이벤트 루프 전체가 멈춤**. 초당 수십~수백 메시지가 들어오는 hub에서 치명적.
- **선택:** hub는 **producer(큐에 put만)**, 별도 worker가 **consumer(sync DB 전담)**. 마 이그레이션 비용이 큰 sync 도메인 코드를 유지하면서, 큐 자체를 자연스러운 백프레서 지점으로 활용.

```
# hub(producer): DB 호출 없이 큐에 넣기만 — 이벤트 루프 보호
try:
    db_queue.put_nowait({"op": "...", "cam_id": cam_id,
    ...})
except asyncio.QueueFull:
    logger.warning(f"queue full, drop cam={cam_id}")    # 손
    실 허용
    update_cache_state(...)    # 캐시는 즉시 갱신 → 알람 즉시성 보장
```

5.2 카테고리별 큐 격리 + drop-on-full 정책

- 카테고리별로 큐·worker를 **1:1 독립**시켜 head-of-line blocking 제거(한 카테고리 지연이 다른 카테고리를 막지 않음).
- 큐가 가득 차면 현재 입력을 **drop**하되 캐시는 갱신 → **eventual consistency**. 영속화 결손은 통계 정확도만 낮출 뿐, 알람의 즉시성에는 영향 없음. (C++ 연산 서버의 백프레서 정책과 의도적으로 동일)

5.3 C++ 단절 내성 — 지수 백오프 + 자동 재구독

- **문제:** 운영 중 C++가 재기동되어도 Python의 모든 워커·hub는 살아 있어야 클라이언트 응답이 끊기지 않음.
- **선택:** WS 재연결에 **지수 백오프**(3 → 6 → ... → 30초 cap)를 적용하고, 재연결 직후 클라이언트가 구독했던 카메라를 **자동 재구독**. 로그 폭주를 막기 위해 **첫 실패만 WARNING**, 재연결 시 INFO로 복귀.

5.4 캐시 계층으로 DB 왕복 차단

- 1초 주기 상태 모니터 루프와 각 hub가 매번 DB를 조회하면 폴링 폭주 → **읽기는 캐시, 쓰기만 DB+캐시 동시** 정책. hub 메시지 처리의 평균 DB 왕복을 ~0으로 낮춤.

5.5 이벤트 상태 머신 — 진행 중 이벤트의 단일 진실

- 진행 중 이벤트를 “**종료 시각이 비어있는 열린 로그**” 하나로 표현하고, 같은 키로 동시에 두 개가 열리지 않도록 **역등 처리**. 별도 상태 플래그 없이 인덱스 효율과 쿼리 단순성을 확보.
- 감지 신호가 일정 시간 끊기면 타임아웃으로 **자동 종료**(좀비 이벤트 방지). 다건을 단일 UPDATE로 달아 N+1 회피.

5.6 이벤트 영상 녹화 — FFmpeg + 좀비 프로세스 방지

- **세그먼트 muxer**로 FFmpeg가 5분 단위 자동 분할(Python이 STOP/START를 반복하지 않음).
- **Windows Job Object(`KILL_ON_JOB_CLOSE`)**: Windows 자식 프로세스는 부모가 죽어도 살아남는 문제가 있어, 운영자가 프로세스를 강제 종료하면 FFmpeg가 좀비로 남음. Job Object에 자식을 넣어 부모 종료 시 OS가 함께 정리하도록 `ctypes` 로 kernel32를 직접 호출.
- 이벤트가 세그먼트 중간에 끝나도 **다음 5분 경계까지 녹화 후 stop** → 모든 파일이 일관된 길이. 경과 시간은 `time.monotonic()` 으로 측정해 시계 보정(NTP)에 영향받지 않음.

5.7 Graceful Shutdown — 우선순위 종료

- FastAPI **lifespan**으로 startup/shutdown을 같은 스코프에 짝지어 부분 실패를 방지.
- 종료 순서를 **현실 세계 부작용 우선**으로 설계: ① 알람 릴레이(사이렌·신호등이 ON으로 남으면 안 됨) → ② 외부 PLC 연결 → ③ 내부 asyncio 워커. 각 단계에 타임아웃 후 강제 cancel fallback을 두어 무한 대기 차단.

5.8 Pydantic 타입 안전성

- Create / Update / Out 스키마를 분리하고 Enum·IP 검증을 적용 → 잘못된 입력을 422/400으로 즉시 거부. 도메인 함수는 valid 데이터만 가정해 방어 코드가 사라짐.

외부 통합(개요): PLC 연동을 위한 Modbus TCP 서버(고정 레지스터 맵 + dirty-check로 보드 부하 감소)와, 결정적 채널 매핑 기반 알람 릴레이 제어(시작 시 전체 OFF로 safety 우선)를 구현했습니다. 세부 매핑·인코딩은 면접에서 설명 가능합니다.

6. 트러블슈팅

좀비 FFmpeg 근절 (Windows Job Object) — 부모 프로세스 강제 종료 시 자식 FFmpeg가 RTSP를 계속 점유하며 좀비로 남는 문제를, `ctypes`로 Win32 Job Object를 생성해 부모 종료와 자식 정리를 OS 레벨에서 결합해 해결.

세그먼트 경계 stop (monotonic clock) — 이벤트 종료 즉시 녹화를 끊으면 마지막 파일이 어중간한 길이로 남는 문제를, “다음 5분 경계까지 녹화 후 stop” 정책 + 단조 시계로 해결해 일관된 재생 경험을 확보.

C++ 재기동 중 자동 복구 — 데이터 소스(C++)가 재기동되어도 관리 서버가 죽지 않고, 재연결 직후 클라이언트 구독을 자동 복구해 클라이언트는 단절을 인지하지 못하도록 설계.

7. 한계 & 개선 로드맵

영역	항목	개선 방향
Python	sync SQLAlchemy	<code>asyncio + asyncpg</code> 비동기 전환으로 <code>to_thread</code> 오버헤드 제거
Python	개별 DB write	worker가 큐에서 배치로 모아 batch INSERT
공통	관측성 부족	Prometheus 메트릭(큐 길이·drop·처리시간), Dead Letter Queue
공통	테스트	도메인 유틸(상태 머신·IOU)의 결정성 단위 테스트
C++	락 경쟁 가시성	카메라별 락 contention 프로파일링 도구
C++	이식성	CMake 기반 Linux 빌드 검증

8. 배운 점

- **이기종 서버의 역할 분리 설계** — 처리량·지연 특성에 따라 책임을 나누고, 두 프로세스가 하나의 운영 가설을 공유하도록 인터페이스와 정책을 일관되게 맞추는 경험.
- C++ 저수준 동시성은 프로젝트에서 재설계 방향을 도출하며 접했고, 이후 데이터 레이스·lock ordering·strand·코루틴·backpressure 등을 **꾸준히 심화 학습**하고 있습니다.